

- `repeat_print(string="hi", count=4)`
- `repeat_print(count=4, string="hi")`

All three of these will print the string 'hi' four times. By providing the name of the parameter we can skip providing values for unnecessary parameter, and we needn't remember the order of the parameters as long as we remember the names.

Classes

Finally we come to object-oriented development using Python with classes. We'll jump right into things with classed by defining a somewhat functional class to represent complex numbers.

- `class Complex:`
- `def __init__(self, real, imag):`
- `self.real = real`
- `self.imag = imag`
- `def __repr__(self):`
- `return '{} + i{}'.format(self.real, self.imag)`
- `def add(self, other):`
- `real_sum = self.real + other.real`
- `imag_sum = self.imag + other.imag`
- `return Complex(real_sum, imag_sum)`

All our class can do is initialise a complex number, and add two complex numbers, but it's enough to understand the basics of classes in Python.

The first is the `__init__` method. This is the constructor of the class, i.e. the function that is called when a new object of this class is created. The parameters that this method gets are the initialisation parameters for this class. Methods in Python are provided an additional parameter (called `self` by convention) that gets the value of the object when called. This is similar to the 'this' variable present in languages like C / C++ etc but feels a lot less magical since it is explicitly visible.

The constructor method simply stores the provided real and imag parameters within the object by adding them as properties of 'self'. They can then be accessed by other methods in this class.

The second interesting method is `__repr__`. This method is designed to returned a string representation of a complex object. If we did not write this